

Robotics Bootcamp

Fall Semester 2025

Bachelor of Global Engineering (BoGE)

Coordinators: Mohamad MROUÉ and Philippe BENABES

Instructors: Clara CIOCAN, Frédéric BOULANGER, Florent LOETE, Tony HANNA, Mohamad MROUÉ, and Philippe BENABES

Technical Support: Carles CATTY

Team: Kenza SEBTI , Rita BERRADA EI AZIZI , Hanaé FROTIER de BAGNEUX, Amaury CAGNOL, Ezer HAMOU



université
PARIS-SACLAY



McGill

Faculty of
Engineering

Objectives and Key Features

Objectives

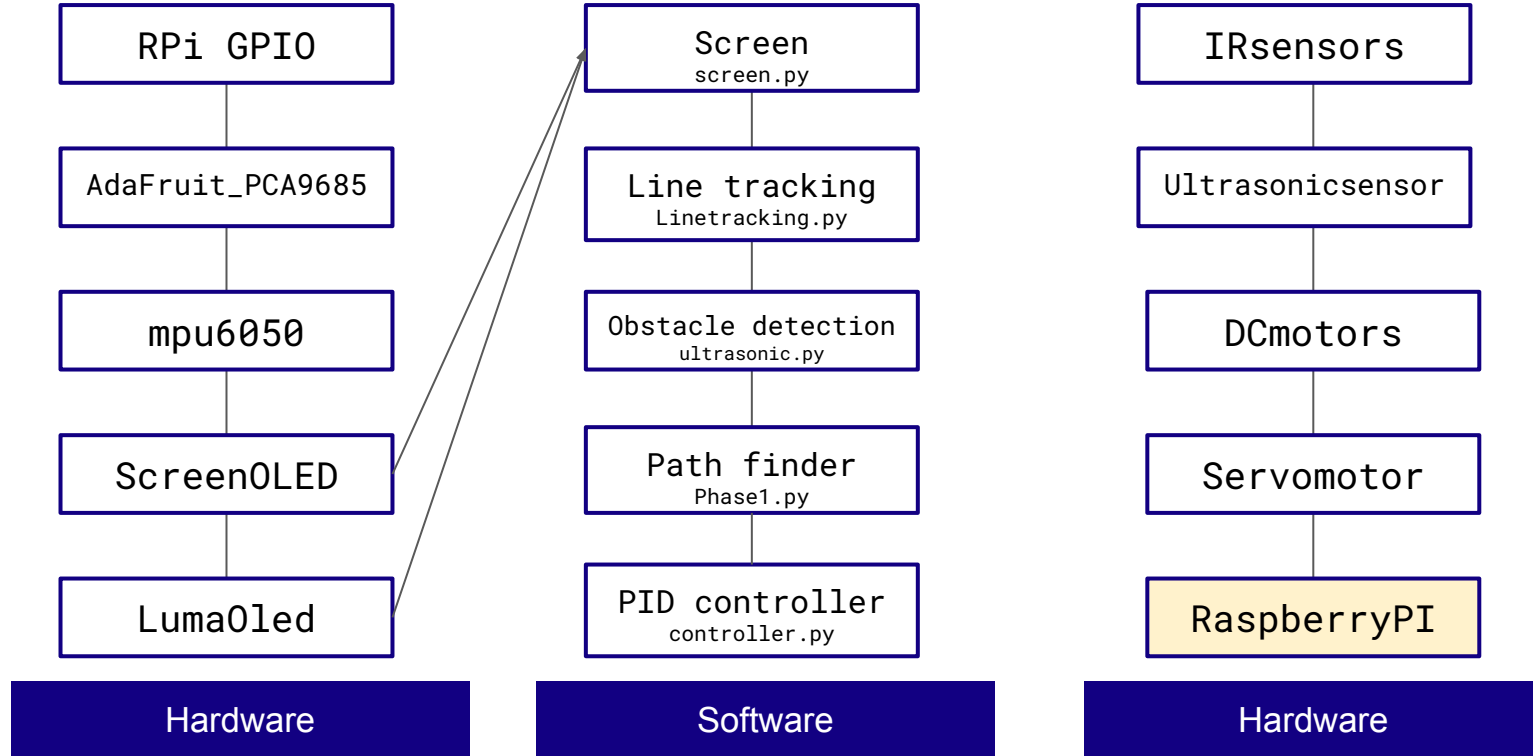
Design and program an autonomous robot capable of reliably following a path and avoiding obstacles



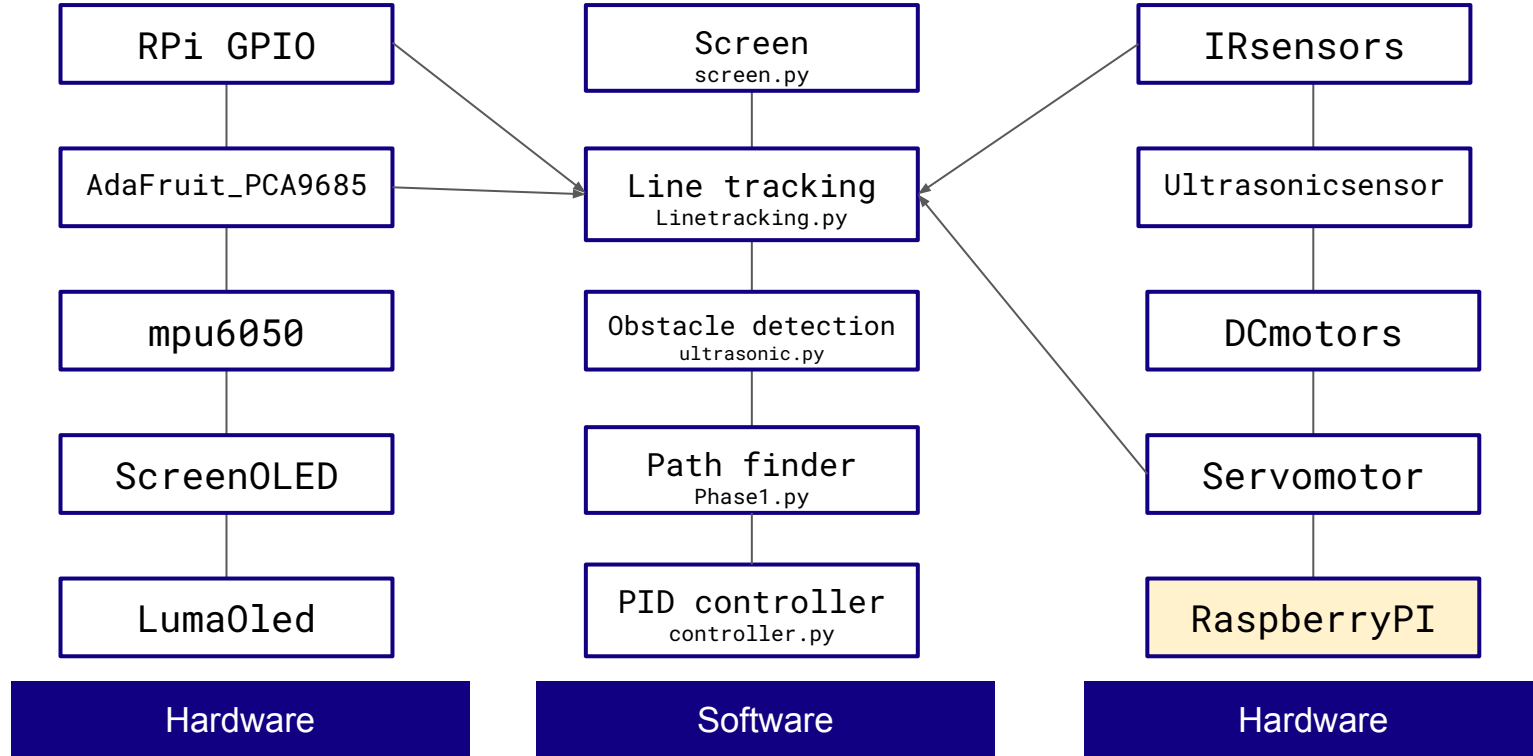
Key Features

- Autonomous line following
- Intersection handling
- Line recovery
- Obstacle avoidance
- Smooth motion control

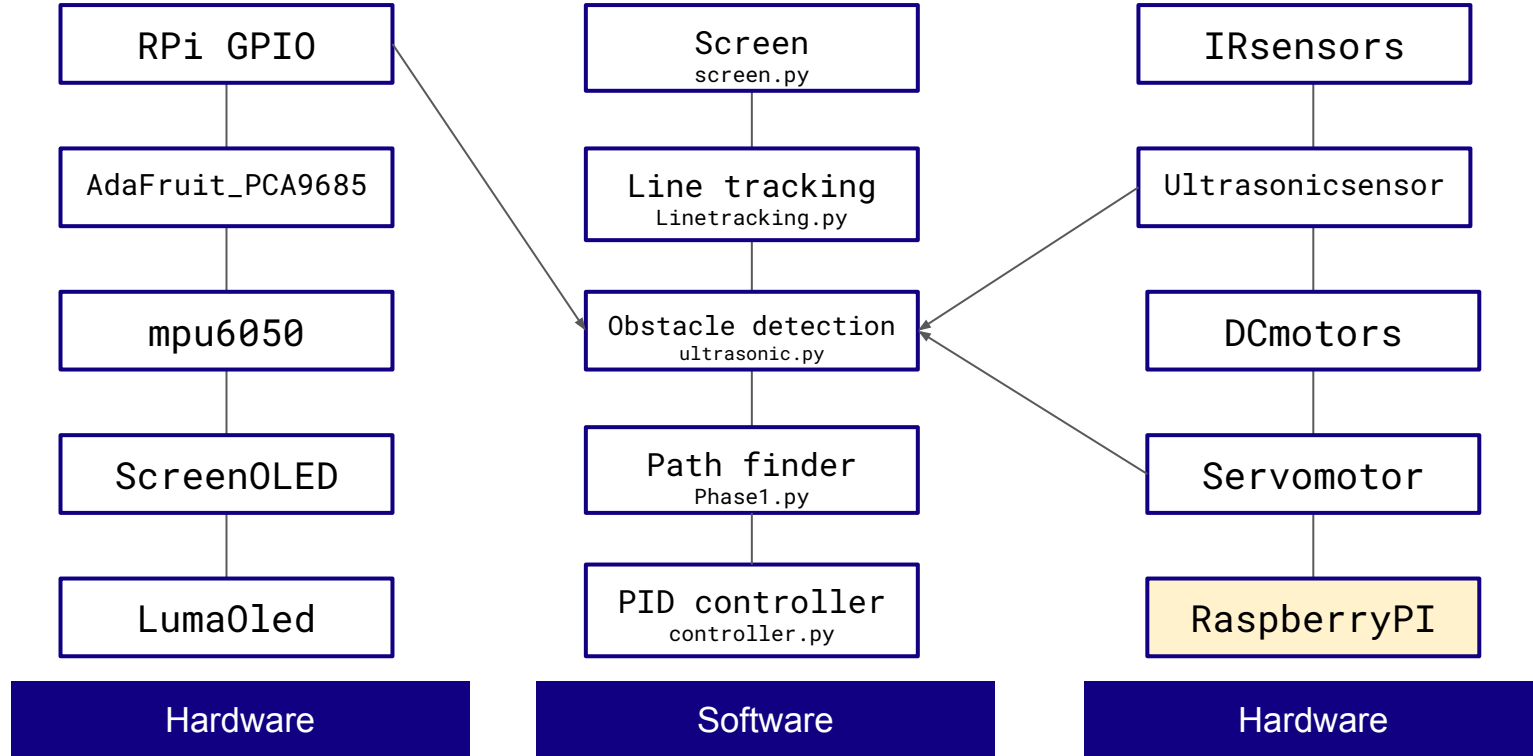
Flowchart illustrating how components hardware and software interact



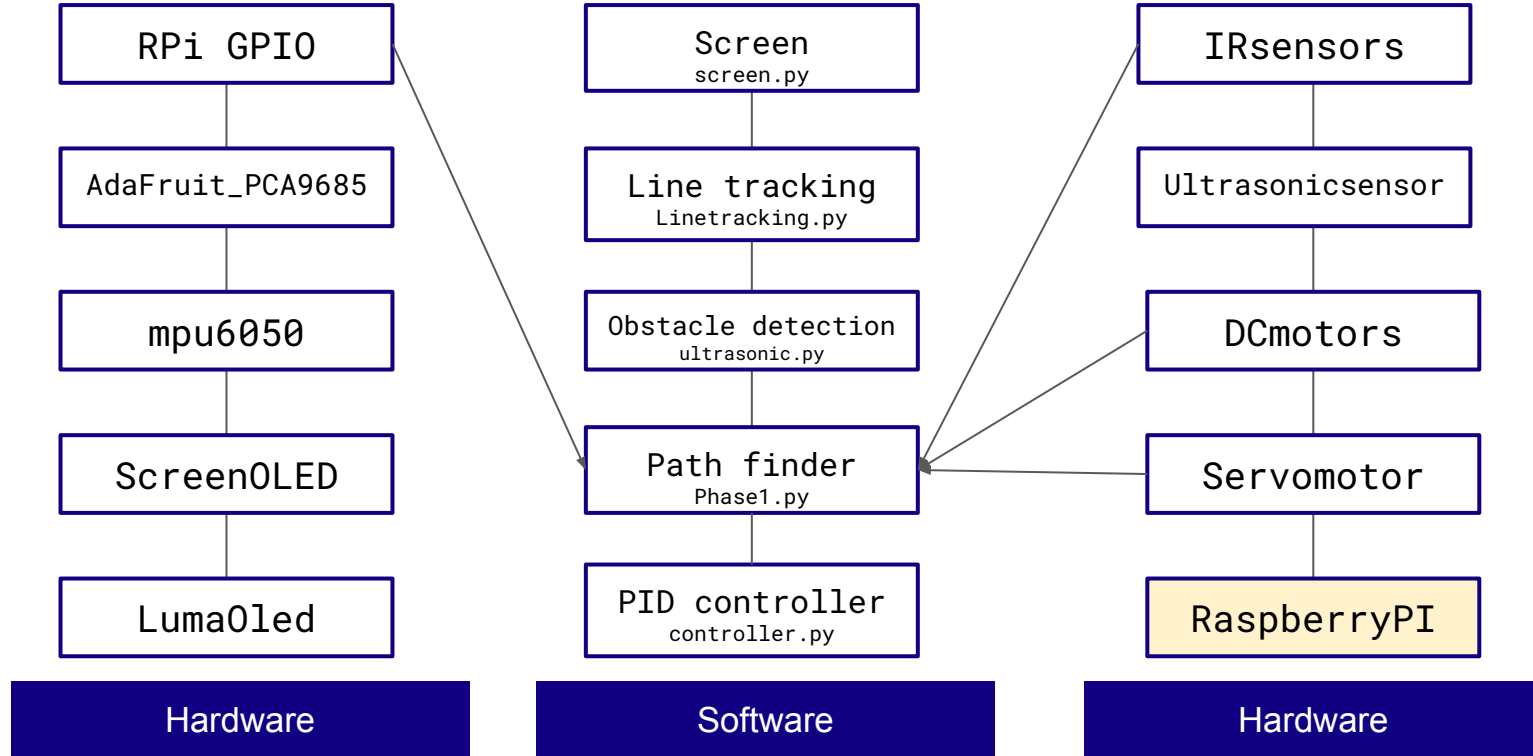
Flowchart illustrating how components hardware and software interact



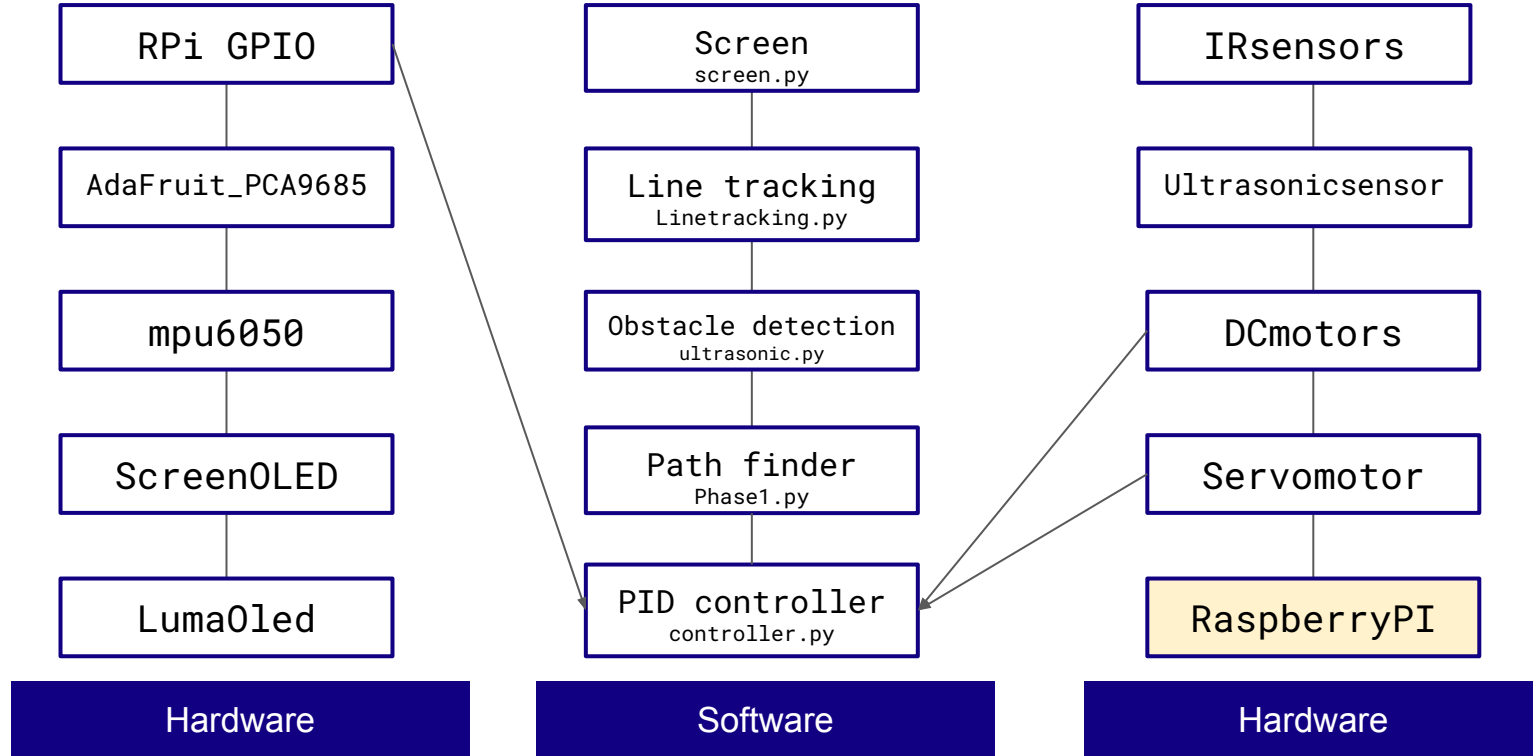
Flowchart illustrating how components hardware and software interact



Flowchart illustrating how components hardware and software interact



Flowchart illustrating how components hardware and software interact



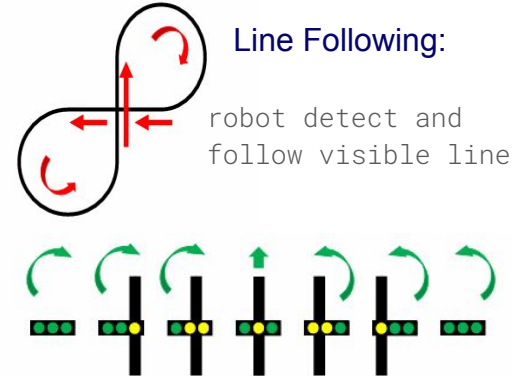
PID

Manhattan

Obstacle Avoidance

A PID controller adjusts the robot's steering

- **Proportional (P)** : Corrects the immediate error. To align robot with the line.
- **Integral (I)** : Accounts for past errors to reduce drift. Corrects consistent offset (bias)
- **Derivative (D)** : Predicts future errors for smoother control. Smooths turns and prevents overshoot



$$P = K_P \cdot \text{Error}$$

$$I = K_I \cdot \text{Sum of Errors}$$

$$D = K_D \cdot (\text{Error} - \text{Previous Error})$$

PID

Manhattan

Obstacle Avoidance

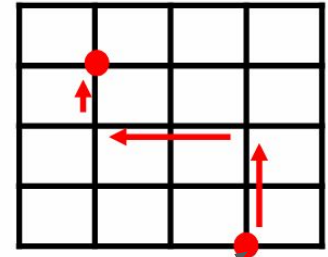
Converts grid coordinates into a **sequence of moves** ("forward", "left", "right", "stop").

- Computes the shortest path on the grid using BFS (Breadth-First Search)
- Uses intersection detection (**IR sensors = 0/0/0**) to trigger decisions.
- Performs accurate 90° turns using a multi-step servo + motor maneuvers.
- Switches between PID line following and intersection actions.

get_path(): from (x_start, y_start) to (x_end, y_end)

Between intersections, **robot.run()** uses PID to keep the robot on the line.

follow_path():
main loop that
mixes PID +
intersection-based
turns until
"stop"



Intersection = 0/0/0 on the sensors → read next command in the path.

PID

Manhattan

Obstacle Avoidance

The code stops at intersections and scans obstacles to determine which directions are safe.

- Determines which directions are **safe to take** with `get_available_paths()` Returns a list of safe directions + a status message ("All clear", "Only LEFT", etc.).
- Repositions on the intersection by reversing before scanning.
- Uses LEDs to show which directions are **blocked** with `indicate_obstacles()`
- Displays distances and available paths on the **OLED screen**.

Stops at intersections and scans front, left, right using the **ultrasonic sensor + head rotation**.



Demonstration

- Follow the circuit with sharp turns (includes a drift correction function).



Follow the circuit with sharp turns

Demonstration

- Follow the circuit with sharp turns (includes a drift correction function).
- Perform a 360° turn using controlled drift for smoother rotation.



Perform a 360°

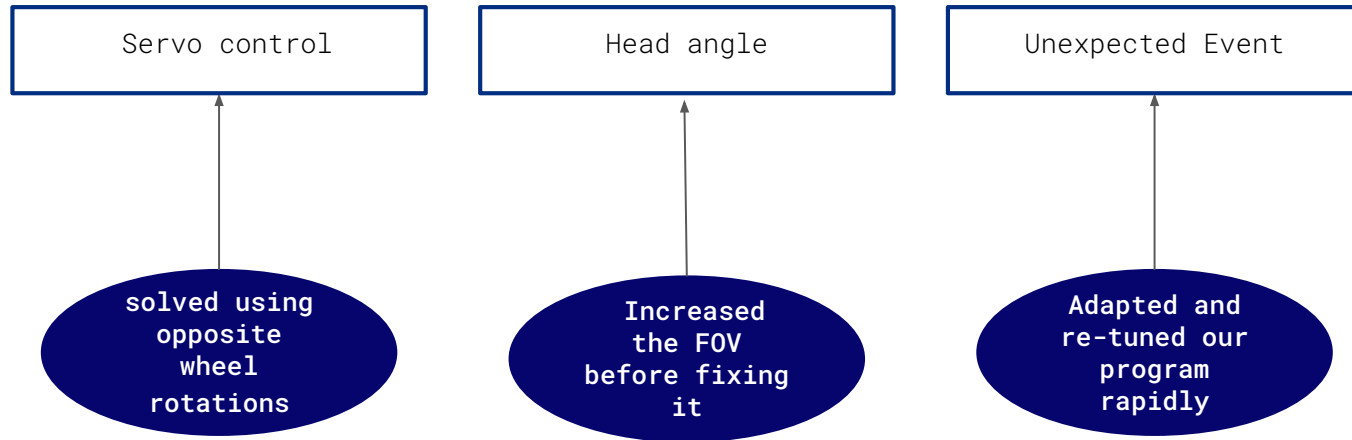
Demonstration

- Follow the circuit with sharp turns (includes a drift correction function).
- Perform a 360° turn using controlled drift for smoother rotation.
- Handle ascending/descending slopes:
 - **Use the MPU6050 to detect ramps steeper than 15° .**



Handle ascending/descending slopes

Challenges & Solutions



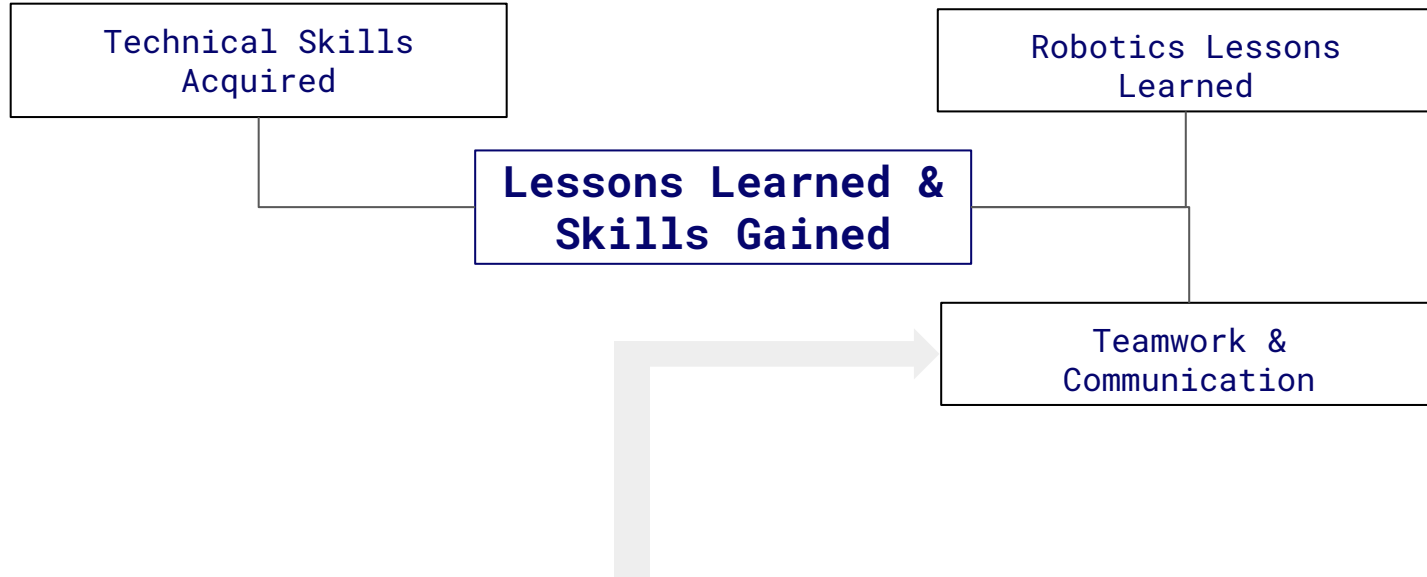


- Built a clean Python architecture using classes (motors, sensors, PID, navigation), improving modularity, readability, and debugging.
- Integration of key sensors: IR (line tracking), ultrasonic + servo (obstacles), MPU6050 (slopes)
- PID controller for smooth and accurate line following.



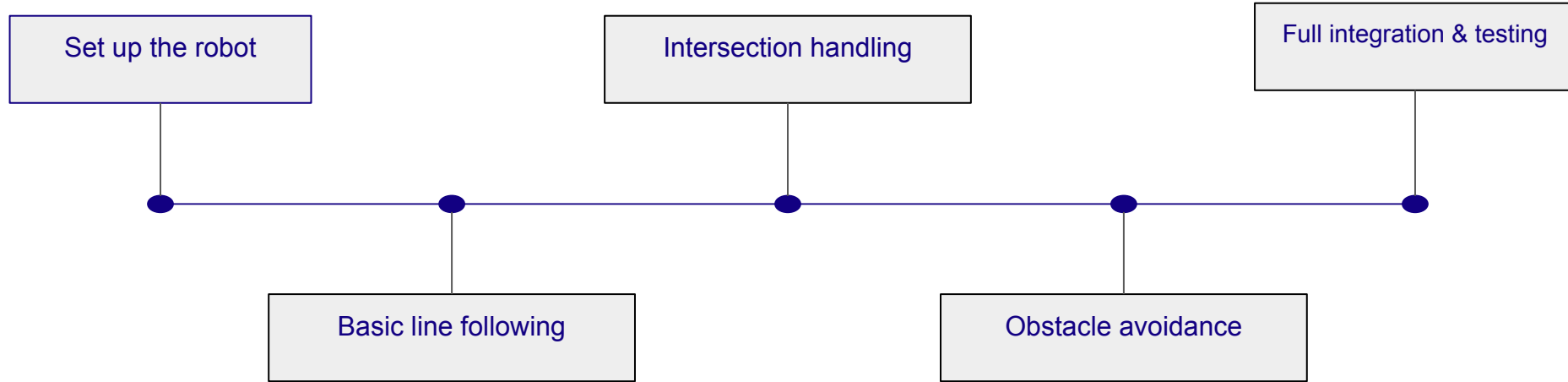
- Understanding of core robotics concepts: real-time feedback loops, sensor fusion, and state management.
- Experience implementing reliable behaviors: line tracking, intersection handling, obstacle detection, and recovery.





- Adopted an iterative routine: **plan** → **implement** → **test** → **refine** workflow throughout the project.
- Developed stronger **problem-solving habits** and a more disciplined approach to debugging.
- Developed the habit of always giving clear and detailed feedbacks for better communication

Summary: Recap the project's achievements



Improvements

Improvements

- Add a small basket/container so the robot can transport items from one point to another
- Add a camera for smarter obstacle detection (choose between reversing, rerouting, or bypassing)

Applications & Future Development

Improvements

- Add a small basket/container so the robot can transport items from one point to another
- Add a camera for smarter obstacle detection (choose between reversing, rerouting, or bypassing)

More mass means the motors must work harder
The PID may need recalibration (new inertia, new braking distance)
Heavy loads could shift the center of gravity

Applications & Future Development

Improvements

- Add a small basket/container so the robot can transport items from one point to another
- Add a camera for smarter obstacle detection (choose between reversing, rerouting, or bypassing)

Use OpenCV on the Raspberry Pi
Detect simple shapes (boxes, walls) or colors
Combine vision with ultrasonic data to make safer decisions

Thank you for your attention

*Kenza SEBTI , Rita BERRADA EI AZIZI , Hanaé FROTIER de
BAGNEUX, Amaury CAGNOL, Ezer HAMOU*

